

CSCE 636 DEEP LEARNING · MODULE 03 · LECTURE NOTES

Optimisation in Practice

Making a non-convex surface cooperate.

MODULE OUTCOMES

- Compare SGD, momentum, and Adam and say when each wins.
- Choose a learning rate and a schedule with evidence.
- Explain why initialisation matters and how it is derived.
- Apply the standard debugging sequence to a failing run.
- Explain what is and is not understood about why this works.

1 The learning rate

NOTHING ELSE MATTERS AS MUCH

Too high and the loss diverges, oscillates, or produces NaN. Too low and it converges so slowly that you conclude the architecture cannot learn the task. Both failures look like model problems and neither is.

The window between the two is frequently less than an order of magnitude, and the right value shifts with batch size, architecture, initialisation, and normalisation — so a value carried over from another project is a guess.

Find it by range test. Start at a very small rate, multiply it by a constant factor each step, and plot the loss against the rate on a log axis. The loss falls, reaches a minimum, and then diverges; **take roughly an order of magnitude below the point of minimum loss** (or just past the steepest descent).

Ten minutes of range test beats a day of guessing, and it should be the first thing done with any new architecture or dataset. It is also the cheapest possible intervention: one short run.

SGD SOMETIMES GENERALISES BETTER THAN ADAM

Adam converges faster in training loss and frequently reaches a worse test loss than carefully tuned SGD with momentum — the effect is most consistently reported in computer vision.

The usual explanation appeals to the noise. SGD's gradient noise is larger and less structured, which biases it toward wide flat minima; flat minima are argued to generalise better because small parameter perturbations change the loss little. **Adam's adaptivity suppresses exactly that noise.**

The explanation is contested — the flatness measure is not reparameterisation-invariant, which is a real objection — **while the empirical pattern is reasonably robust.** It is worth holding the fact and the explanation separately.

So in practice: use AdamW to get something working quickly, and try tuned SGD with momentum if the last point of accuracy matters. And measure, because it does not hold everywhere — transformers generally prefer Adam, and the reasons for that are also not fully settled.

3 Initialisation

$$\text{Var}(\text{out}) = n_{\text{in}} \cdot \text{Var}(w) \cdot \text{Var}(\text{in}) \Rightarrow \text{Xavier: } \text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}}) \quad \text{He: } \text{Var}(w) = 2/n_{\text{in}}$$

For a linear layer with n inputs, assuming independence, the output variance is the input variance times the fan-in times the weight variance. **Setting the weight variance to preserve activation variance across layers keeps the signal from growing or shrinking exponentially with depth** — which is Module 02 §3's problem addressed at initialisation. **Xavier** balances the forward and backward directions, for symmetric activations. **He's extra factor of two compensates for ReLU zeroing half the activations**, which halves the variance — a derivation that takes two lines and makes the constant meaningful rather than magical. **These are derived, not tuned**, and getting the scale wrong by a factor of two per layer compounds exponentially.

ZERO INITIALISATION DOES NOT WORK, AND NEITHER DOES LARGE

All zeros is the symmetry failure: every unit in a layer computes the identical function and therefore receives the identical gradient, so they update identically and remain identical forever. **The layer has exactly one effective unit regardless of its width**, and no amount of training fixes it. **Symmetry must be broken by randomness.**

Too large is the saturation failure: activations are pushed into the flat regions of bounded functions (where the gradient is nearly zero) or grow without bound for unbounded ones — and the gradients follow in either case.

Biases are usually fine initialised at zero, because the weights already break the symmetry. **The one common exception is initialising the final layer's bias to the base rate**, which saves the network from spending its first epochs learning the class prior — a small and genuinely useful trick on imbalanced problems.

And modern practice initialises the residual branch near zero so that each block starts as approximately the identity. **The network then begins as a shallow one and deepens as training proceeds**, which means adding depth costs nothing at step one — an elegant resolution of the tension between depth and trainability.

4 Debugging a failing run

- 1 **Overfit one batch.** Take eight examples and train until the loss is near zero. **If you cannot, there is a bug and nothing else matters** — a shape error, a detached gradient, a wrong loss, labels misaligned with inputs. **This takes under a minute and catches the overwhelming majority of problems**, and it is the single most valuable habit in this course.
- 2 **Check the loss at initialisation.** For k balanced classes under cross-entropy it should be about $\ln(k)$ — 2.30 for ten classes. **If it is not, the output layer, the loss, or the label encoding is wrong**, and that is a thirty-second check for a bug that otherwise takes hours.

- 3 **Run the learning rate range test (§1).**
- 4 **Look at the data.** Actually look at it — render the images after augmentation, print the tokenised text, listen to the audio, **as the model receives it**. Augmentation bugs and preprocessing bugs are invisible in the loss and obvious on screen.
- 5 **Then tune. Everything before this step is bug-finding; only now is it optimisation**, and conflating the two is why people tune hyperparameters for a week against a shape error.

WHY THIS WORKS AT ALL IS NOT FULLY UNDERSTOOD

The loss surface is non-convex, extremely high-dimensional, and has an enormous number of critical points. Gradient descent has no theoretical guarantee of finding anything good, and Module 02 of CSCE 633 noted that convexity was what made the classical methods safe.

It works anyway, reliably, across architectures and datasets. The current partial understanding: in very high dimensions most critical points are saddles rather than poor local minima (a bad minimum requires the Hessian to be positive in *every* direction, which becomes improbable), and the many minima that do exist appear to be of broadly similar quality.

And the optimiser appears to regularise implicitly — among the many parameter settings that fit the training data exactly, gradient descent reliably finds low-complexity ones, which is the mechanism behind double descent (CSCE 633 Module 03 §4).

This is an active research area rather than settled knowledge. Be suspicious of confident mechanistic explanations — including the two in this callout, which are the current consensus and have both been challenged. **The honest position is that the practice is far ahead of the theory**, which is unusual in this program and worth noticing rather than papering over.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Smith — Cyclical Learning Rates and the LR range test (free)	The §1 range test , from the paper that introduced it. Short and immediately applicable.
Loshchilov & Hutter — Decoupled Weight Decay Regularization (free)	The AdamW fix of §2 , with the explanation of what Adam's weight decay was actually doing.
He et al. — Delving Deep into Rectifiers (free)	The §3 initialisation derivation , including the factor of two.
Karpathy — A Recipe for Training Neural Networks (free)	The §4 procedure, in full . The most practically useful single page in this course.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Run a learning rate range test** on your project and plot the curve. Choose a rate from it.
- 2 Train with rates 10× above and below that choice, and plot all three curves.
- 3 **Compare constant, step, and cosine schedules** at equal total steps.
- 4 **Add warmup** to a transformer-style model and show what happens without it.
- 5 Compare SGD, SGD+momentum, Adam, and AdamW on the same problem. **Report training and test loss for each.**
- 6 **Derive He initialisation** for ReLU by hand.
- 7 Initialise a network to all zeros and demonstrate that a layer has one effective unit.
- 8 **Initialise residual branches near zero** and compare early training against standard initialisation.
- 9 **Overfit a single batch of eight examples** on your project, and record how long it took to get working.
- 10 Check your loss at initialisation against $\ln(k)$ and fix it if it disagrees.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does the learning rate matter most, and how should it be found?
- 2 Explain explore-then-exploit and why schedules decay.
- 3 Compare six optimisers on what each adds.
- 4 What bug does AdamW fix?
- 5 Why might SGD generalise better than Adam, and how confident should you be?
- 6 Derive the He initialisation factor.
- 7 Why does zero initialisation fail?
- 8 Give the five debugging steps in order and say which catches most bugs.
- 9 What is and is not understood about why this works?